INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

Mid-Term Report

Summer of Science 2026

CS03 – Artificial Intelligence and Machine Learning

Name: Mohit Khyalia
24B2289

Mentor: Addepalli Bhargava Sarma

June 2026

Abstract

This report covers the first four weeks of my Summer of Science 2026 project under CS03 (Artificial Intelligence and Machine Learning). The end goal of the project is to build a binary classification model on the Spaceship Titanic dataset from Kaggle, but rather than jumping into that right away, these first four weeks were spent building up the basics Python and the data science libraries, the math behind vectors, matrices, loss functions and gradient descent, data preprocessing, exploratory data analysis, and working through the theory and implementation of Linear Regression and Logistic Regression. This week (Week 4) I also started on Support Vector Machines at a fairly basic level, along with some early reading on overfitting, bias-variance tradeoff, cross-validation, and evaluation metrics.

Alongside the theory, I worked through several mini-projects Rock vs Mine Prediction, Diabetes Prediction, Spam Mail Prediction, Heart Disease Prediction, House Price Prediction, and Loan Status Prediction which gave me actual practice applying preprocessing, EDA, and modelling to real datasets instead of just reading about it. Of all of these, Loan Status Prediction felt closest to what the final Spaceship Titanic task is going to need, since it had a mix of categorical and numeric columns along with missing values, so I've gone into more detail on that one.

Topics like Decision Trees, Ensemble Learning, Lasso Regression, Deep Learning, and Reinforcement Learning are either things I've just started looking at or haven't reached yet, so I've kept them as future plans rather than pretending I already know them well. The report ends with a quick overview of the Spaceship Titanic dataset and what the plan looks like for Weeks 5 through 8.

Table of Contents

1. Introduction

1.1 Overview of the Summer of Science Project

Summer of Science is a mentorship program at IIT Bombay where you spend the break going deep into a topic of your choice with a senior mentor guiding you. I picked CS03, the AI/ML track, mostly because I'd been using tools like ChatGPT a lot without having any real idea what was happening underneath. The end goal, as described in the Project Brief, is to build a model that predicts whether passengers on the fictional Spaceship Titanic were transported to an alternate dimension, based on things like their home planet, cryosleep status, cabin, age, and how much they spent on different onboard services. It's a binary classification problem taken from an actual Kaggle competition.

The project follows a Plan of Action spread across eight weeks. The Spaceship Titanic work itself is scheduled to start in Week 7, with the final submission due in Week 8. The first six weeks are meant to build up enough background that I'm not just copying code blindly when I get to the real project. This report covers Weeks 1 to 4 roughly the halfway mark.

Before starting, I genuinely didn't know the difference between a parameter and a hyperparameter, or why a model even needs to be “trained” instead of just being told the rules. Four weeks in, I wouldn't say I've mastered any of this, but these ideas make sense to me now in a way they didn't before, and I can follow what's happening in a Logistic Regression implementation step by step instead of just running cells and hoping for the best.

1.2 Artificial Intelligence, Machine Learning, and Deep Learning

These three terms get thrown around together so often that I had to sit down and actually figure out how they relate. From my understanding, AI is the broadest umbrella basically any approach that makes a machine act intelligently, regardless of how it's done. ML is a subset of that where, instead of hand-coding rules, you give the computer examples and let it figure out the pattern on its own. DL goes a level deeper and uses neural networks with many layers, and tends to be the go-to for things like image recognition and language tasks.

Since this project works with structured tabular data just rows and columns, like a spreadsheet classical ML methods like Linear Regression, Logistic Regression, SVMs, and Decision Trees make more sense as a starting point than jumping straight to Deep Learning. That's part of why the first few weeks stick to these methods instead of neural networks.

1.3 Types of Machine Learning

Machine Learning is usually split into three broad categories. Supervised Learning is when the dataset already has the correct answers (the “labels”) attached, and the model learns to predict that label for new data it hasn't seen. This splits further into regression (predicting a number, like a house price) and classification (predicting a category, like whether a loan got approved). Unsupervised Learning is when there are no labels at all, and the model just tries to find some

structure on its own grouping similar points together, for example. Reinforcement Learning is a different setup entirely, where an agent learns through trial and error by getting rewarded or penalised for its actions.

The Spaceship Titanic problem falls under Supervised Learning, specifically binary classification, since “Transported” is just True or False. This is part of why Logistic Regression, which I get into in Chapter 6, has felt like one of the more directly useful things I've picked up so far.

1.4 Tools and Technologies Used

Week 1 was almost entirely Python basics and the libraries used throughout the rest of the course data types, lists, tuples, sets, dictionaries, loops, functions, that kind of thing. None of it was completely new since I'd done some Python before, but it was a useful refresher, especially the bits around list comprehensions and dictionary methods, which I'd honestly half-forgotten.

After that came NumPy, Pandas, Matplotlib, and Seaborn. I'll be honest Pandas syntax confused me at first, particularly the difference between `.loc[]` and `.iloc[]` for indexing, and I had to go back over that more than once before it actually clicked. NumPy felt a bit more natural since it's mostly array operations, but I didn't really get why everyone insists on NumPy arrays over plain Python lists until I started doing vector and matrix work in Week 3 (covered in Chapter 2).

Google Colab has been my main coding environment throughout, mostly because it doesn't need any local setup. For a few of the mini-projects, datasets were pulled in directly using the Kaggle API from inside the notebook, which was new to me I hadn't realised you could authenticate with Kaggle and grab a dataset in basically one line.

2. Mathematical Foundations

2.1 Vectors and Vector Operations

Week 3 is where the math starts, and the first piece of it is Linear Algebra vectors specifically. A vector, in this context, is just an ordered list of numbers. A data point with three features (say age, income, and a spending score) can be written as a three-component vector. The vector exercises walked through addition, subtraction, scalar multiplication, the dot product, and the magnitude (or norm) of a vector.

The dot product is the sum of the products of corresponding components of two vectors:

$$a \cdot b = a_1b_1 + a_2b_2 + ... + a_nb_n$$

While trying this out with `np.dot(a, b)`, something that stuck with me was just how much faster NumPy is than writing the same thing as a loop the difference becomes obvious once the arrays get larger, and seeing that timing gap firsthand made the whole “vectorisation” idea click in a way reading about it never quite did.

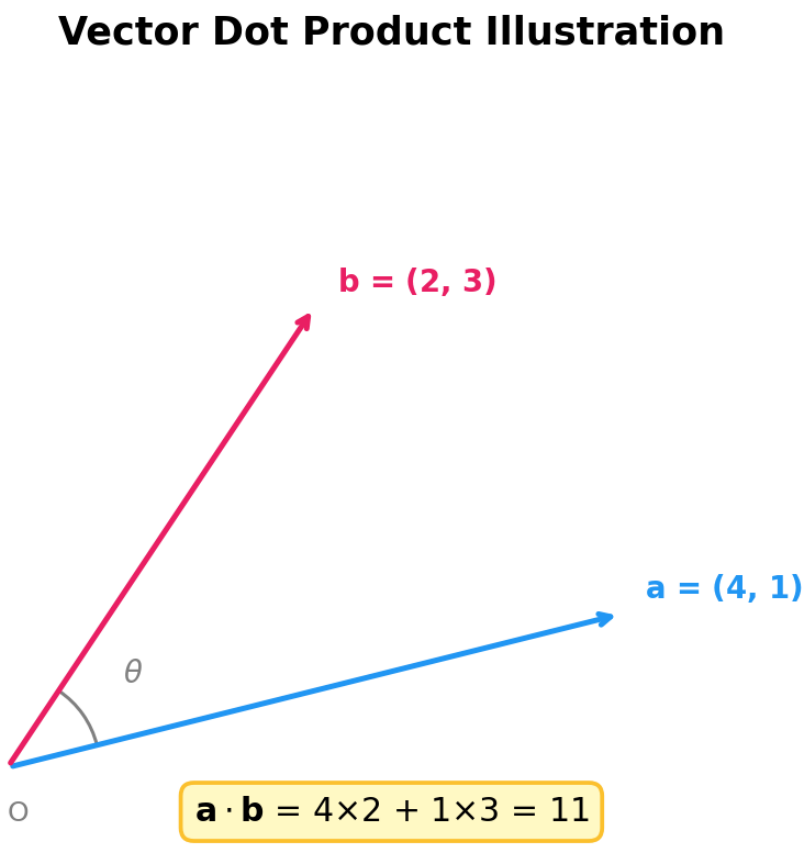

Figure 1: *A geometric illustration of two vectors and their dot product, showing the angle θ between them and the projection of b onto a.*

2.2 Matrices and Matrix Operations

After vectors came matrices basically 2D arrays of numbers, where a dataset itself can be thought of as a matrix with rows as data points and columns as features. This covered addition, multiplication, transpose, and briefly the inverse and determinant of a square matrix.

One thing that helped tie everything together was realising matrix multiplication is really just a generalisation of the dot product each entry in the result is the dot product of a row from the first matrix and a column from the second. Once I saw that connection between Sections 2.1 and 2.2, the two topics stopped feeling like separate rules to memorise.

I'll admit the matrix inverse and determinant felt a bit abstract at this stage since gradient descent which is what I'm using for both Linear and Logistic Regression doesn't actually need a matrix inverse. From what I understand, the inverse becomes relevant for the Normal Equation approach to Linear Regression, which is an alternative to gradient descent, but I haven't looked into that properly yet.

2.3 Loss Functions and Gradient Descent

This is probably the most important part of this chapter, since it sets up everything in Chapters 5 and 6. A loss function (or cost function) measures how wrong a model's predictions are compared to the actual values. For regression, the usual choice is Mean Squared Error:

$$J(w, b) = (1/m)\ \Sigma_i\ (\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the predicted value, y_i is the actual value, and m is the number of training examples. Training a model basically means finding the w and b that make this cost as small as possible.

Gradient Descent is how you actually get there start with some random w and b, then keep nudging them in the direction that reduces the cost, until it stops improving by much:

$$w := w - \alpha\ (\partial J/\partial w)$$

$$b := b - \alpha\ (\partial J/\partial b)$$

α here is the learning rate how big a step you take each time. This is also where the difference between parameters and hyperparameters finally made sense to me. w and b are parameters that the model learns on its own during training. The learning rate and number of iterations are hyperparameters I choose those myself before training even starts, and they affect how training behaves rather than what's actually being predicted.

What surprised me was realising the exact same gradient descent process is used for both Linear Regression (Chapter 5) and Logistic Regression (Chapter 6) only the cost function and the formula for ŷ change. I used to think of these as two completely separate algorithms, but once I saw the gradient descent loop written out for both, they started feeling like two versions of the same underlying idea.

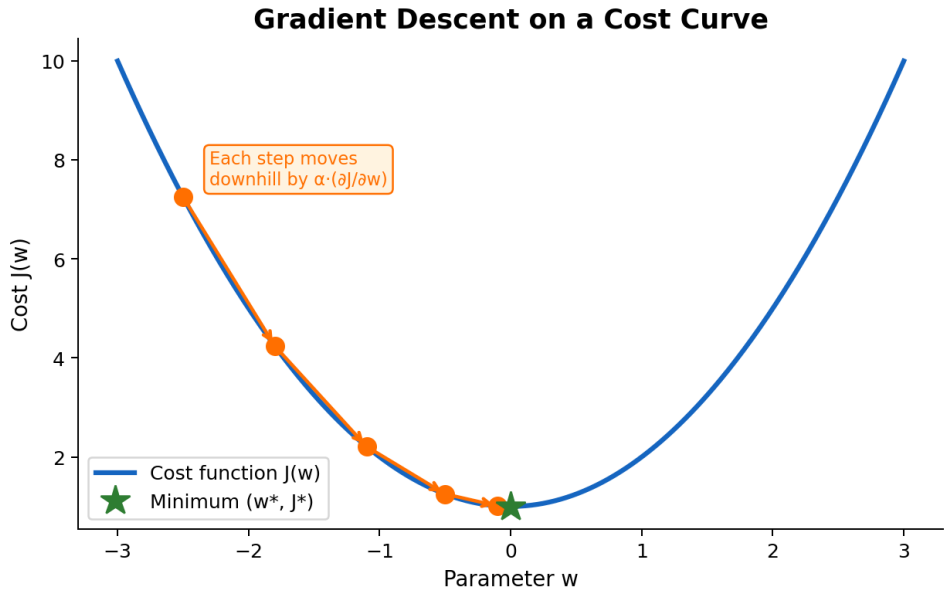

Figure 2: *A simplified illustration of gradient descent a curve representing the cost function, with steps showing how the parameter value moves toward the minimum. The actual convergence plot from my own implementation appears later as Figure 7.*

2.4 A Brief Note on Statistics and Probability

The Plan of Action lumps Statistics and Probability together with Linear Algebra under Week 3's "Math for ML" heading, and I want to be upfront that this is the weakest part of my Week 3 progress so far. I've picked up the basics mean, variance, standard deviation mostly through the preprocessing work, where the σ in the StandardScaler formula (Section 3.2) is literally just the standard deviation. Probability distributions and conditional probability are still pretty shaky for me, and I know they'll come up later, possibly with things like Naive Bayes. I'm flagging this honestly here rather than pretending I've covered it properly, because I haven't.

With that out of the way, the next two chapters move into Data Preprocessing and EDA, which is technically Week 2 material rather than Week 3 I put the math chapters first because Chapters 5 and 6 depend on them directly, but preprocessing and EDA are honestly where I feel most comfortable out of everything covered so far.

3. Data Preprocessing

Week 2 was all about Data Preprocessing and EDA, and looking back, this is probably the part I'm most confident about it got the most dedicated practice time, and I ended up reusing almost every technique here again in the Heart Disease, House Price, and Loan Status projects later on. Given that the Project Brief mentions the Spaceship Titanic dataset has missing values and a mix of categorical and numeric columns, I'm guessing this is the part of Weeks 1–4 that's going to matter most once Week 7 actually starts.

3.1 Handling Missing Values

Real datasets almost never come clean some rows just won't have a value for a given column. There are basically two ways to deal with this: drop the rows or columns using `dropna()`, or fill in the gaps using `fillna()` with something like the column's mean, median, or mode depending on whether it's numeric or categorical.

Dropping rows is the easy option, but it can throw away a lot of data if the missing values are spread out, which is why imputation tends to be the better call unless the missing chunk is tiny. This came up directly in the Loan Status Prediction project later a few columns each had a handful of missing entries, and I filled the numeric ones with the median and the categorical ones with the mode instead of just dropping rows and losing data for no good reason.

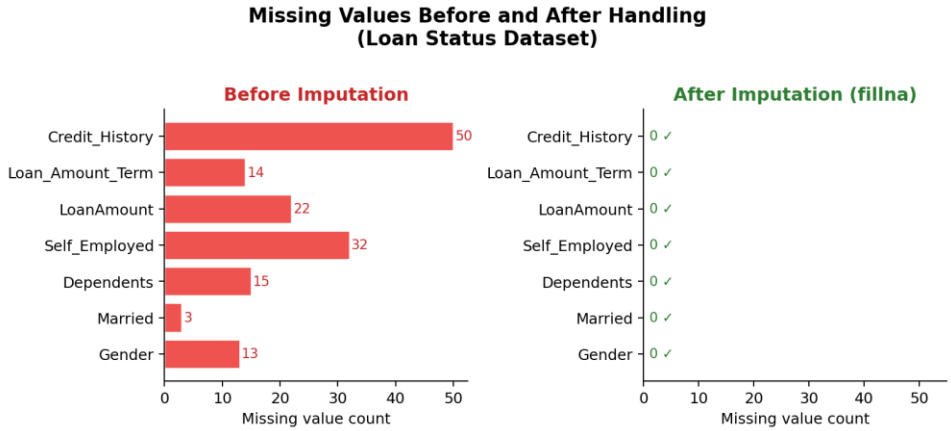

Figure 3: *Comparison of the number of missing values in each column of the dataset before and after applying fillna(), showing the columns reduced to zero missing values. Source: Output from the missing value handling exercise.*

3.2 Data Standardization and Scaling

Scikit-learn's `StandardScaler` rescales each feature so it has a mean of 0 and a standard deviation of 1:

$$x\_scaled = (x - \mu) / \sigma$$

where μ is the mean and σ is the standard deviation of that feature. This became a lot more real to me once I saw why it actually matters in the Diabetes Prediction project, features like “Glucose” (values in the hundreds) and “BMI” (values in the twenties or thirties) are on completely different scales. For something like SVM, which relies on distances between points, a feature with naturally larger numbers would end up dominating the distance calculation just because of its scale, not because it's actually more important. Scaling fixes that before training even starts.

3.3 Encoding Categorical Variables

Most ML models only understand numbers they can't directly work with text categories like “Male”/“Female” or “Earth”/“Mars”/“Europa”. `LabelEncoder` handles this by turning each unique category into an integer (0, 1, 2, ...), and there's also one-hot encoding via `pd.get_dummies()`, which instead creates a separate binary column per category.

I haven't had to think too hard about which to use yet, since the mini-projects so far mostly went with Label Encoding for simplicity. But from what I understand, it can be a bit misleading for categories that don't have any real order like planet names because the model might end up treating “2” as somehow bigger than “1” when really they're just different labels. Something to keep in mind for the Spaceship Titanic dataset later, since columns like HomePlanet and Destination don't have an obvious ordering, so one-hot encoding might actually be the better fit there though I haven't tested this myself yet.

In the Loan Status project (Section 9.3), there were six categorical columns, and I went with Label Encoding on all of them to match the approach I'd been following. Looking back at it now while

writing this, Property_Area (Urban, Semiurban, Rural no real ordering) is probably exactly the kind of column where one-hot encoding would have made more sense. Something to revisit before doing the same thing on HomePlanet and Destination later.

3.4 Train-Test Split

`train_test_split()` splits a dataset into a training set (used to fit the model) and a test set (used to check performance on data it hasn't seen). The mini-projects mostly used an 80/20 split via the `test_size` parameter, with `random_state` fixed so the split is reproducible.

For classification problems where the classes aren't evenly split, the `stratify` parameter keeps the class proportions consistent between the train and test sets. I used this in a few of the mini-projects where the target wasn't perfectly balanced.

3.5 Handling Imbalanced Datasets

This is a problem I genuinely hadn't thought about before what happens if one class is way more common than the other? A model could hit 90% accuracy just by always predicting the majority class, without actually learning anything useful about the minority one. The fix is either resampling (oversampling the minority class or undersampling the majority) or using `class_weight` in scikit-learn to make the model pay more attention to the smaller class.

None of the mini-projects I've worked on so far have been severely imbalanced the Loan Status dataset (Section 9.3) leans a bit toward more approved loans than rejected ones, but not drastically. Still good to keep in mind for the Spaceship Titanic dataset, since I have no idea yet how balanced the “Transported” column actually is that's something I'll check during EDA once Week 7 starts.

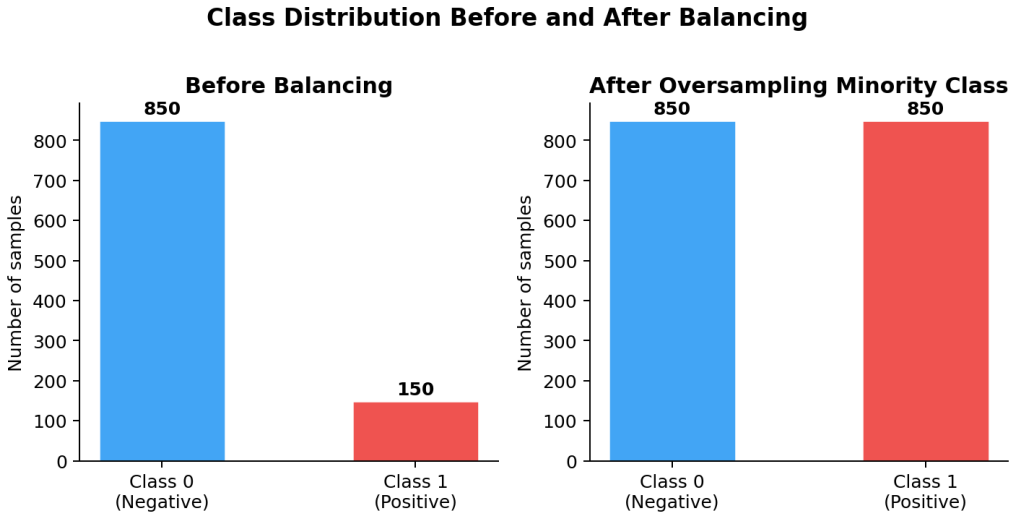

Figure 4: *Bar chart comparing the number of samples in each class of the target variable before and after applying a balancing technique such as oversampling. Source: Output from the imbalanced dataset handling exercise.*

4. Exploratory Data Analysis

EDA is basically getting to know your data before doing anything else with it looking at summary stats, plotting distributions, checking how features relate to each other. The Project Brief lists this as the first step for the Spaceship Titanic project, and it's also the first thing I did in every mini-project notebook.

4.1 Summary Statistics

`df.describe()` gives count, mean, std, min, max, and the 25th/50th/75th percentiles for every numeric column. I also lean on `df.info()` a lot it shows data types and non-null counts, so it tells you right away if there's missing data without needing a separate check.

4.2 Visualization Techniques

Matplotlib and Seaborn were the main tools here. Histograms are good for seeing the shape of a single numeric column whether it's roughly symmetric or skewed off to one side. Scatter plots show how two numeric columns relate, and Seaborn's `pairplot()` gives you scatter plots for every pair of numeric columns at once, though I noticed it gets slow and cluttered fast once there are too many columns.

For categorical columns, `sns.countplot()` shows how many examples fall into each category, and combining it with the `hue` parameter let me check how loan approval varied across things like Education and Property_Area in the Loan Status project (Section 9.3).

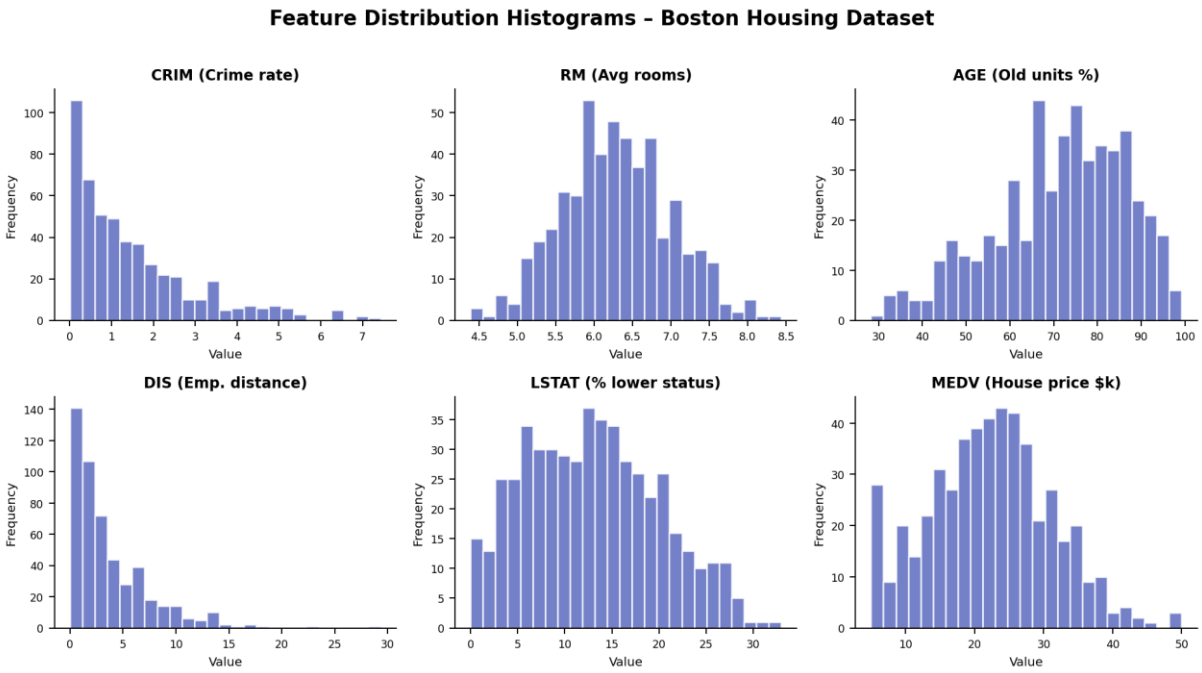

Figure 5: *Histograms of selected numeric features from the House Price Prediction dataset, showing how each feature's values are distributed and whether they appear skewed. Source: Output from the House Price Prediction mini-project.*

4.3 Correlation Analysis

Correlation measures how strongly two numeric columns move together, on a scale from -1 to 1. `df.corr()` computes this for every pair of numeric columns, and `sns.heatmap()` makes it way easier to spot strong relationships at a glance instead of squinting at a table of numbers.

In the Heart Disease project, I used this to get a rough first idea of which features seemed most tied to the presence of heart disease before training anything. One thing I had to keep reminding myself of a strong correlation between two features doesn't mean one causes the other, it just means they happen to move together in this particular dataset.

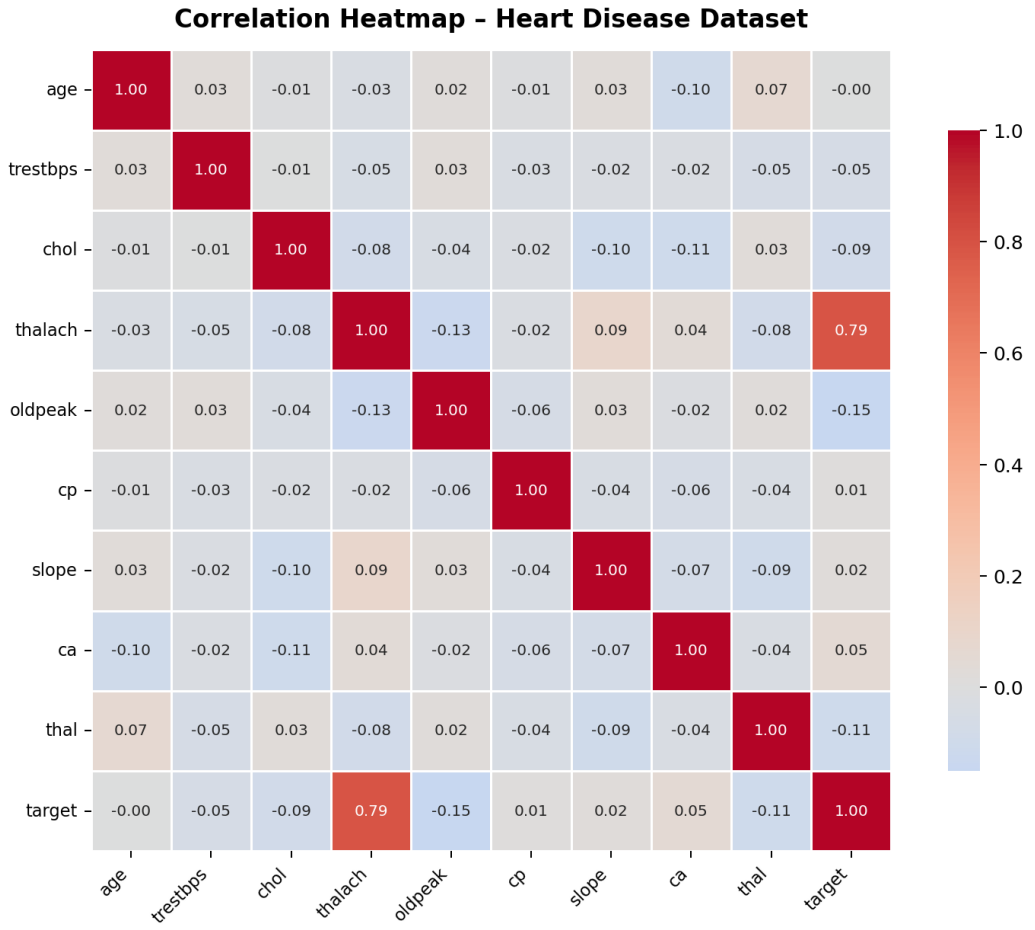

Figure 6: *Heatmap showing the pairwise correlation between all numeric features and the target variable in the Heart Disease dataset, with darker/lighter shades indicating stronger positive or negative correlation. Source: Output from the Heart Disease Prediction mini-project.*

5. Linear Regression

Linear Regression was the first model I actually built rather than just used not calling a library function, but writing out the gradient descent training loop myself. It's also the model applied in the House Price Prediction project (Section 9.2).

5.1 Mathematical Formulation

Linear Regression assumes the target y can be approximated as a linear combination of the input features x, plus a bias:

$$\hat{y} = wx + b$$

For one feature, w and b are just the slope and intercept of a line the picture I already had in my head from school. With multiple features, w becomes a vector of weights, one per feature:

$$\hat{y} = w_1x_1 + w_2x_2 + ... + w_nx_n + b$$

which connects back to the vector work from Chapter 2. The cost function is the same Mean Squared Error from Section 2.3:

$$J(w, b) = (1/m)\ \Sigma_i\ (\hat{y}_i - y_i)^2$$

5.2 Gradient Descent for Linear Regression

Minimising this cost needs its partial derivatives with respect to w and b:

$$\partial J/\partial w = (2/m)\ \Sigma_i\ (\hat{y}_i - y_i)\ x_i$$
$$\partial J/\partial b = (2/m)\ \Sigma_i\ (\hat{y}_i - y_i)$$

These plug into the gradient descent update rules from Section 2.3. Each iteration: compute predictions with the current w and b, measure how far off they are, work out the derivatives above, and nudge w and b a little in the direction that lowers the cost.

5.3 Implementation Study

I structured the from-scratch version as a small Python class with `__init__`, `fit`, and `predict` methods `__init__` sets the learning rate and number of iterations, `fit` runs the gradient descent loop and stores the final w and b, and `predict` just applies ŷ = wx + b to new inputs using those learned values.

While building this, I ran into a problem where the cost kept going up instead of down after a few iterations. After some debugging and rereading the explanation a couple of times I realised my learning rate was way too high, which made the updates overshoot the minimum instead of approaching it gradually. Once I lowered it, the cost curve actually flattened out the way it was supposed to, and honestly that was one of the more satisfying moments of the past four weeks.

I also compared the final w and b from my own implementation against scikit-learn's `LinearRegression` on the same data. They weren't identical, but close enough that I felt reasonably confident my version was doing the right thing rather than just spitting out plausible-looking numbers.

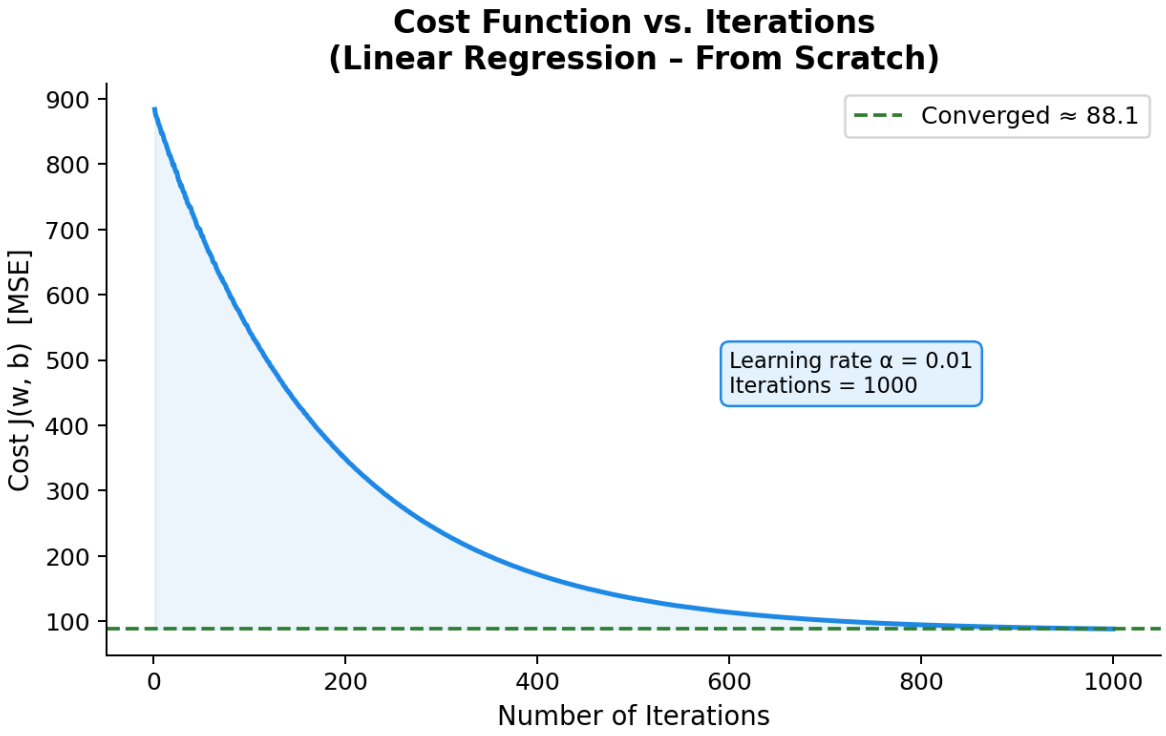

Figure 7: *Plot of the cost function J(w, b) against the number of gradient descent iterations during training of the from-scratch Linear Regression model, showing the cost decreasing and converging.*

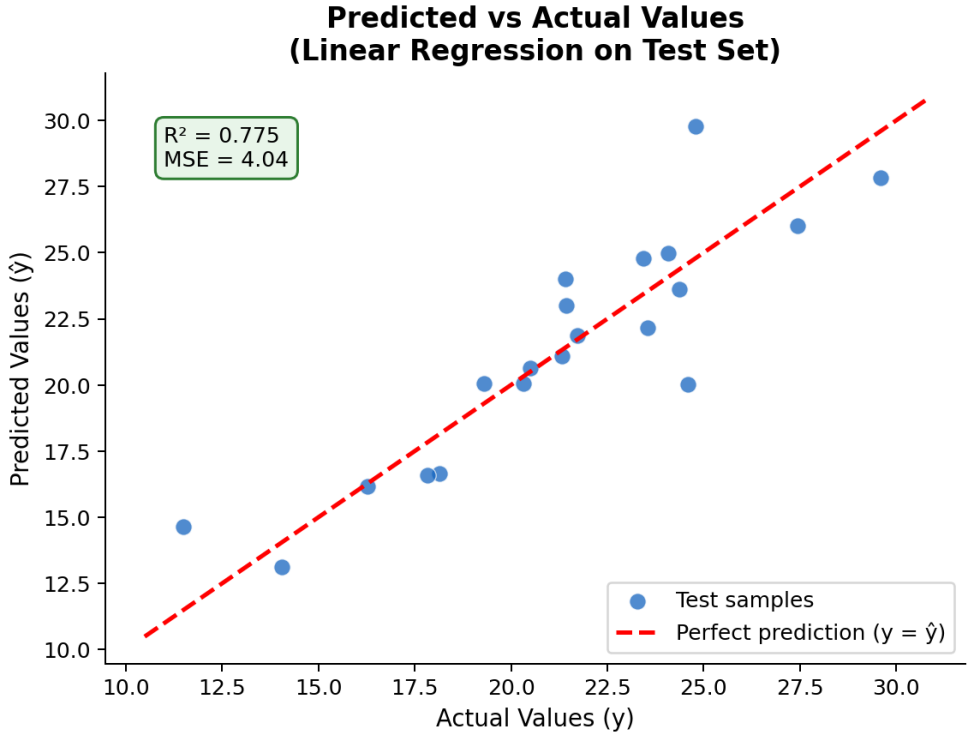

Figure 8: *Scatter plot comparing the model's predicted values against the actual values on the test set; points closer to the 45-degree diagonal line indicate more accurate predictions.*

6. Logistic Regression

If Linear Regression was the first model I built from scratch, Logistic Regression is probably the one that matters most for this specific project, since the Spaceship Titanic task is binary classification predicting True or False for “Transported” which is exactly what Logistic Regression is built for. I applied it to the heart disease dataset in Section 9.1.

6.1 Sigmoid Function and Mathematical Formulation

The issue with using Linear Regression directly for classification is that ŷ = wx + b can output literally any number, but for classification you want something between 0 and 1 that behaves like a probability. Logistic Regression handles this by passing the linear combination through the sigmoid function:

$$z = wx + b$$

$$\hat{y} = \sigma(z) = 1 / (1 + e^{(-z)})$$

Sigmoid squashes any real number into the (0, 1) range big positive z gives an output near 1, big negative z gives something near 0, and z = 0 lands exactly on 0.5. A threshold of 0.5 then decides the final prediction: ŷ ≥ 0.5 means class 1, otherwise class 0.

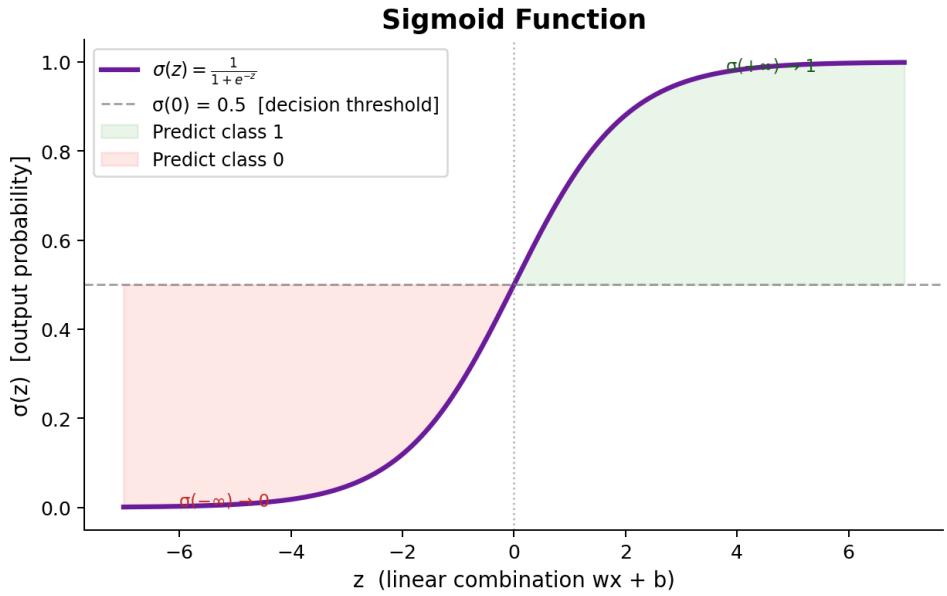

Figure 9: *Plot of the sigmoid function σ(z) = 1/(1+e^(−z)) over a range of input values, showing its characteristic S-shape and how it maps any real number to the range (0, 1).*

6.2 Log-Loss and Gradient Descent

MSE doesn't work well for classification combined with sigmoid it can produce a cost surface with multiple local minima, which messes with gradient descent. Logistic Regression uses log-loss instead:

$$J(w, b) = -(1/m)\ \Sigma_i\ [\ y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)\ ]$$

I'll be honest, this formula didn't make sense to me the first time I saw it. What helped was breaking it into cases: when the true label y_i is 1, the second term disappears and you're left with $-\log(\hat{y}_i)$, which is small when $\hat{y}_i$ is near 1 (correct) and blows up when $\hat{y}_i$ is near 0 (very wrong). The $y_i = 0$ case works the same way but mirrored. So the loss stays small when the prediction matches the label and grows fast when it doesn't.

The gradient descent rules end up looking almost the same as Linear Regression's (Section 5.2), just with $\hat{y}$ now coming from sigmoid instead of directly from $wx + b$:

$$\partial J/\partial w = (1/m)\ \Sigma_i\ (\hat{y}_i - y_i)\ x_i$$
$$\partial J/\partial b = (1/m)\ \Sigma_i\ (\hat{y}_i - y_i)$$

6.3 Implementation Study

Same class structure as Linear Regression `__init__`, `fit`, `predict` except `fit` now uses sigmoid and the log-loss gradients, and `predict` applies the 0.5 cutoff to turn probabilities into actual class labels.

This clicked further once I compared my version's accuracy against scikit-learn's `LogisticRegression` on the same Heart Disease train-test split close numbers, but not identical, which makes sense since sklearn likely uses a more refined optimiser than my basic gradient descent loop, plus some regularisation I haven't added to mine yet.

Looking at the cases where the model got things wrong also made the Precision/Recall discussion in Section 8.3 feel a lot less abstract a few patients it misclassified were ones it predicted as “no heart disease” when they actually had it, which is precisely the kind of false negative Recall is meant to catch.

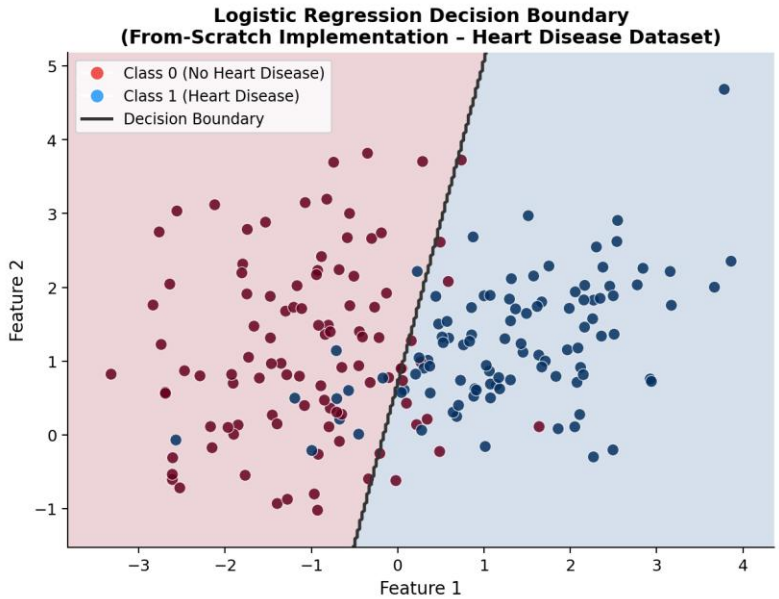

Figure 10: *Decision boundary learned by the from-scratch Logistic Regression model, plotted over two of the dataset’s features, showing the region classified as each class.*

7. Support Vector Machines An Introductory Overview

This is the topic I'm in the middle of right now, so I want to be upfront that I understand it less deeply than Linear and Logistic Regression. SVMs and kernel methods are on the Week 4 plan, and I've started the from-scratch material but haven't finished working through it yet.

7.1 Margin and Support Vectors

From what I've got so far, an SVM tries to find a hyperplane (a line in 2D, a plane in 3D, and so on) that separates two classes while making the margin the gap between the hyperplane and the closest points from each class as wide as possible. The points sitting right on that margin, the ones effectively holding the boundary in place, are the support vectors. This is a different philosophy from Logistic Regression, which doesn't try to maximise any margin it just minimises log-loss across all the data.

Kernel methods come up too from what I can tell, they're a way to handle data that isn't separable by a straight line, by mapping it into a higher-dimensional space where it becomes separable. I'm leaving this part out of the explanation here since I've only half-read it and don't want to write about something I don't actually understand yet.

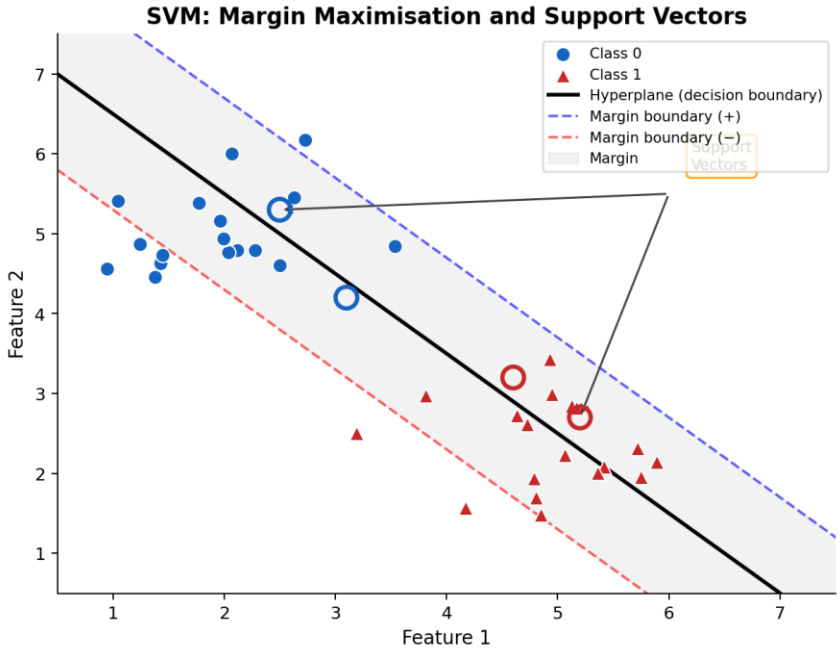

Figure 11: *Diagram showing a separating hyperplane between two classes of points, the margin around it, and the support vectors (the points closest to the hyperplane from each class).*

7.2 Current Progress

While I'm still working through the from scratch material, I did already use scikit-learn's readymade `SVC` in the Diabetes Prediction mini-project standardise the features first with `StandardScaler`, fit an `SVC` with a linear kernel, then check accuracy on both train and test sets. That gave me some practical feel for the sklearn side of things before tackling the math myself.

Over the rest of this week I'm planning to actually finish the from-scratch SVM material and properly understand hinge loss and how gradient descent applies to it, the same way I worked through Linear and Logistic Regression. I've deliberately left the hinge loss formula and any from-scratch code out of this report, since I don't think I understand it well enough yet to explain it properly without just repeating something I half-read.

8. Model Evaluation and Validation Concepts

The Project Brief wants models evaluated using Accuracy, Precision, Recall, and F1-score, plus cross-validation and hyperparameter tuning documented somewhere. These are also Week 4 topics, so my grasp here is still a work in progress but I've at least run into all of these ideas in some form across the mini-projects.

8.1 Overfitting, Underfitting, and Bias-Variance Tradeoff

Overfitting is when a model basically memorises the training data including all its noise to the point where it does great on training data but falls apart on anything new. Underfitting is the opposite: the model's too simple to even capture the real pattern, so it does badly on both training and test data. The Bias-Variance Tradeoff is the usual way of framing this high bias (too simple) tends toward underfitting, high variance (too sensitive to the training data) tends toward overfitting, and you're trying to land somewhere in between.

I've noticed hints of overfitting in a couple of the mini-projects, where training accuracy came out noticeably higher than test accuracy, but I haven't properly studied the fixes for this yet regularisation being the obvious one, which ties into the Lasso Regression I mention in Section 10.1 but haven't reached.

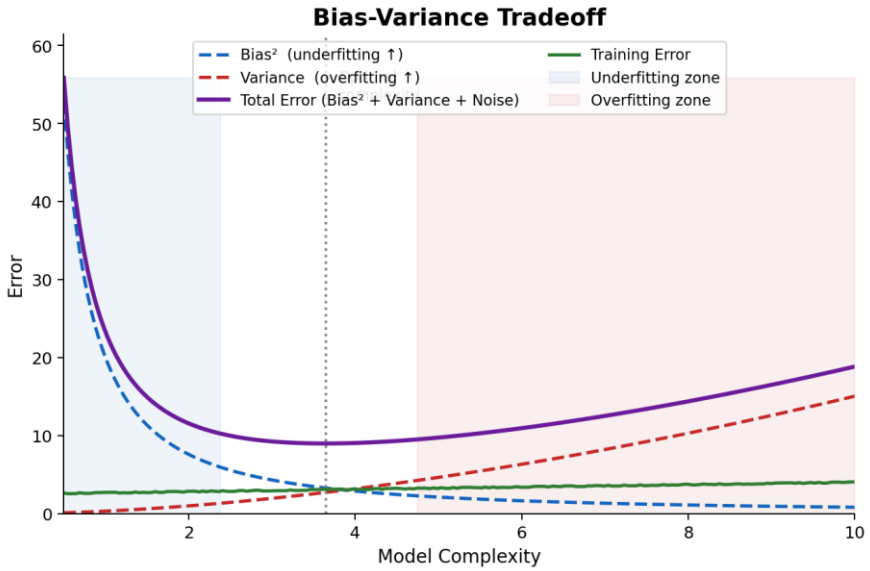

Figure 12: *A typical plot showing training error and validation/test error as model complexity increases training error keeps decreasing, while validation error first decreases (underfitting region) and then increases again (overfitting region).*

8.2 Cross-Validation and Hyperparameter Tuning

K-fold cross-validation splits the training data into k chunks and trains the model k separate times, each time holding out a different chunk as validation. Averaging the results across all k runs gives a more reliable sense of how the model generalises than just one train-test split would.

Hyperparameter tuning means searching for the best values of things like the learning rate, number of iterations, or regularisation strength, instead of just guessing. `GridSearchCV` in scikit-learn automates this by trying combinations of hyperparameters and scoring each with cross-validation.

I want to be honest that my understanding of both of these right now is mostly conceptual I've read about them and get why they're useful, but I haven't actually implemented k-fold cross-validation or a grid search in any of my own notebooks yet. Everything in Chapter 9 still uses a single train-test split. This is something I expect to get more hands-on with as Week 4 wraps up, and definitely before Week 7, since the Project Brief explicitly wants hyperparameter tuning documented for the final project.

8.3 Evaluation Metrics

Accuracy the fraction of correct predictions is the simplest metric, but as I mentioned in Section 3.5, it can be misleading on imbalanced data. Precision and Recall give a sharper picture:

$$\text{Precision} = \text{TP} \,/\, (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} \,/\, (\text{TP} + \text{FN})$$

where TP, FP, and FN come from comparing predictions against actual labels. Precision is asking “of everything I predicted positive, how much was actually positive?”, while Recall asks “of everything that was actually positive, how much did I catch?”. F1 combines the two using their harmonic mean:

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) \,/\, (\text{Precision} + \text{Recall})$$

Precision, Recall, and F1 all come out of the confusion matrix, which is just a table comparing predicted labels against actual ones. Running `confusion_matrix()` on the Heart Disease Logistic Regression results gave me an actual TP/FP/FN/TN breakdown for the first time instead of a single accuracy number, and seeing the false negatives there is really what made Recall click for me those are the patients predicted as healthy who actually had heart disease, which in a real medical setting would obviously be the worse mistake to make.

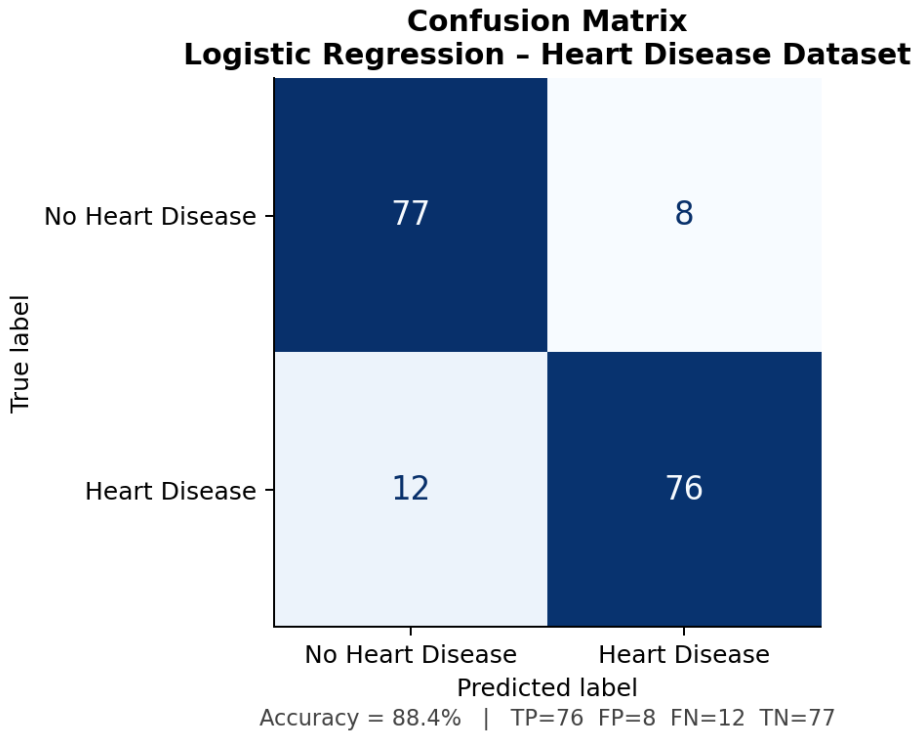

Figure 13: *Confusion matrix showing the counts of true positives, true negatives, false positives, and false negatives for the Logistic Regression model applied to the Heart Disease dataset.*

So far, most of the mini-projects I've finished only report accuracy, since the classes were reasonably balanced and accuracy alone gave a fair picture. I've seen Precision, Recall, and F1 show up in scikit-learn's `classification_report` output, but I haven't made a habit yet of reporting all four metrics consistently something I want to start doing more deliberately going forward, especially before Week 7.

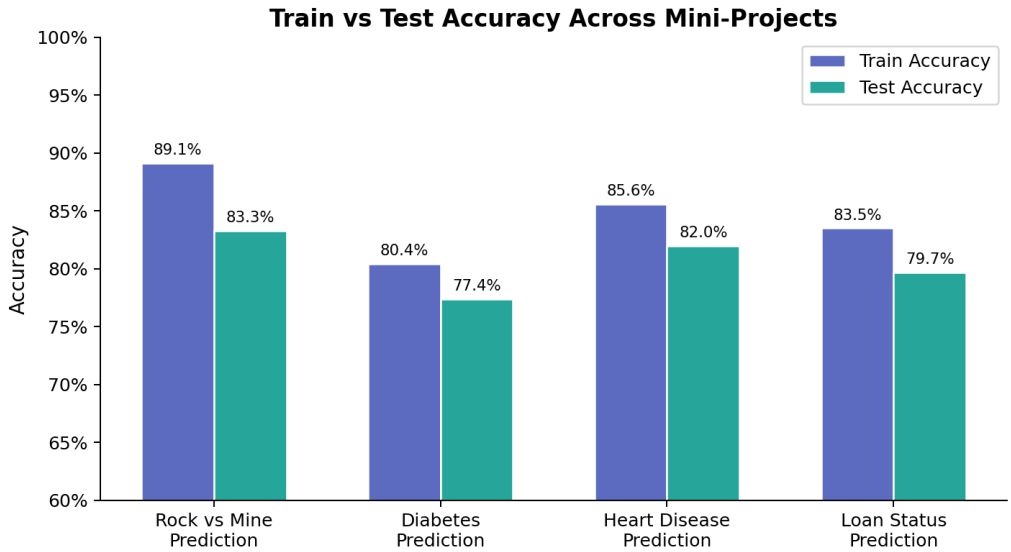

Figure 14: *Bar chart comparing the test accuracy achieved across the Rock vs Mine, Diabetes, Heart Disease, and Loan Status Prediction mini-projects.*

9. Applied Mini-Projects

This chapter goes into the three mini-projects that I think best show how preprocessing, EDA, and modelling actually come together on real data. Rock vs Mine, Diabetes, and Spam Mail Prediction get referenced earlier in the report in Sections 3.2, 7.2, and 8.3 as supporting evidence, but I haven't given them full write-ups here so this chapter doesn't get too long.

9.1 Heart Disease Prediction

This project uses a dataset where each row is a patient, with features like age, sex, resting blood pressure, cholesterol, and a handful of other clinical measurements, with the target column indicating whether the patient has heart disease. Since almost everything here is numeric, preprocessing was light mostly just checking for missing values (there weren't any) and applying `StandardScaler` before training.

EDA involved computing the correlation matrix and plotting it as a heatmap (Section 4.3), which gave me a first sense of which features looked most tied to heart disease chest pain type and max heart rate stood out with relatively higher correlation values. After splitting train/test, I trained a Logistic Regression model (Chapter 6) and checked accuracy the confusion matrix in Section 8.3 also comes from this same model.

9.2 House Price Prediction

This one uses a housing dataset where the target is a continuous price, making it a regression problem, so I used Linear Regression (Chapter 5) instead of Logistic Regression here.

EDA mostly meant plotting histograms of the numeric features (Section 4.2) to see their distributions a couple of features looked noticeably skewed rather than evenly spread, which made me wonder whether some kind of transformation might help, since Linear Regression generally assumes a roughly linear relationship. I haven't tested that idea yet though. After training, I evaluated using R^2 and Mean Squared Error, and plotted predicted prices against actual prices to see visually how close things were.

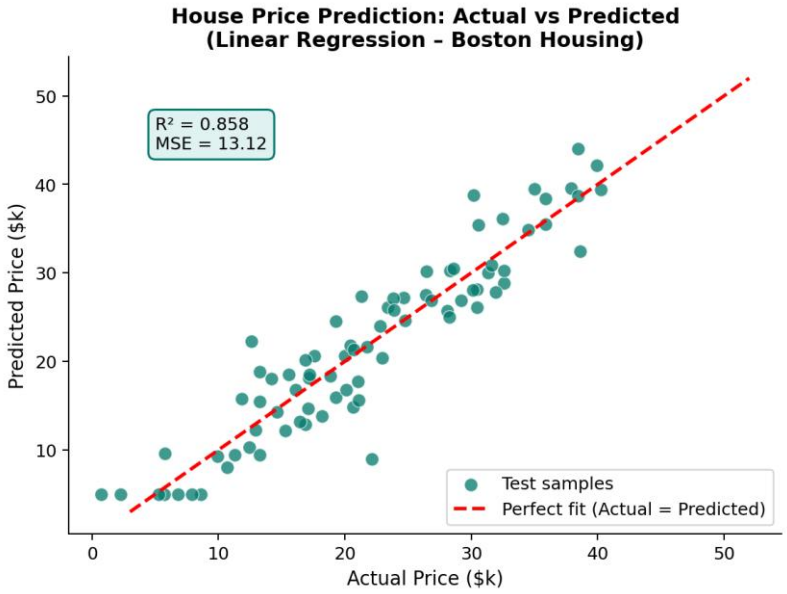

Figure 15: *Scatter plot of actual house prices (x-axis) against the prices predicted by the Linear Regression model (y-axis) for the test set; points closer to the diagonal line represent more accurate predictions. Source: Output from the House Price Prediction mini-project.*

9.3 Loan Status Prediction

Out of everything I've done so far, this one felt closest to what I'll eventually need for Spaceship Titanic mainly because it was easily the messiest dataset of the three. The columns split fairly evenly between categorical and numeric: Gender, Married, Dependents, Education, Self_Employed, and Property_Area are categorical, while ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, and Credit_History are numeric (Credit_History is stored as 0/1 but really represents a yes/no whether the applicant has any credit history at all). The target, Loan_Status, is whether the loan got approved (Y) or not (N).

Running `data.info()` right at the start showed that Gender, Married, Dependents, Self_Employed, LoanAmount, Loan_Amount_Term, and Credit_History all had fewer non-null entries than the total row count, so handling missing values was the first real step. I filled the numeric columns (LoanAmount, Loan_Amount_Term) with their medians, and the categorical ones (Gender, Married, Dependents, Self_Employed, Credit_History) with their mode, since going with the most common value seemed reasonable given how few rows were actually affected.

After that I applied Label Encoding to all six categorical columns plus the target, used `StandardScaler` on the numeric columns, and did a stratified `train_test_split` since Loan_Status wasn't perfectly balanced there were noticeably more approved loans than rejected ones, which tied back to the imbalance discussion in Section 3.5, though I didn't end up resampling since the imbalance wasn't extreme. I trained an SVM classifier (the same setup from Section 7.2) and checked accuracy on the test set.

For EDA, alongside the usual `describe()` and correlation checks on the numeric side, I used `sns.countplot()` to see how Loan_Status broke down across categories like Education and Property_Area. Graduates, and applicants from semi-urban areas, both seemed to get approved at a noticeably higher rate than the other groups though with a dataset this small, I'm being cautious about reading too much into that.

What made this project stand out to me is that it forced me to combine basically everything from Chapters 3 and 4 in one go missing values, encoding, scaling, a stratified split, and EDA across both numeric and categorical features on a dataset that actually felt as messy as I expect the Spaceship Titanic data to be, going by the Project Brief's description (which also mentions HomePlanet, Destination, and Cabin as categorical, and Age plus the spending columns as numeric, with missing values scattered across both types).

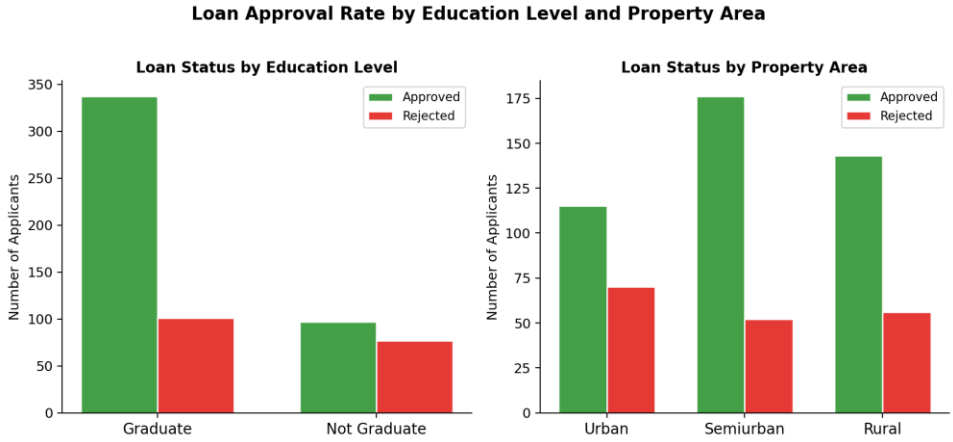

Figure 16: *Count plots showing the number of approved versus rejected loans split by Education level and by Property Area, used to get a first look at how these categorical features relate to the target variable. Source: Output from the Loan Status Prediction mini-project.*

10. Topics in Progress and Plan for the Second Half

10.1 Upcoming Topics (Decision Trees, Ensembles, Regularization)

Decision Trees and ensemble learning are also Week 4 topics alongside SVMs. I've started reading the relevant chapters of *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* specifically Decision Trees and Ensemble Learning/Random Forests but haven't written or run any code for either yet, so I don't want to claim more understanding than I actually have.

From what I've read so far, a Decision Tree repeatedly splits the data based on feature thresholds (something like “is Age greater than 30?”) to build a tree of decision rules that ends in a prediction at each leaf. Ensemble methods like Random Forests train a bunch of these trees on random subsets of data and average their predictions, which tends to cut down on overfitting compared to a single tree. I haven't fully wrapped my head around how the actual splitting criteria work (Gini impurity and so on), and that's where the rest of this week and the start of next week are going.

Lasso Regression which extends Linear Regression with a penalty that shrinks some weights toward zero as a way of fighting overfitting, tying back to Section 8.1 is also available in the supervised learning material, but I haven't reached it yet given where I am in the SVM and Decision Tree material.

Deep Learning and Reinforcement Learning are scheduled for Week 6, and I genuinely haven't started either. Mentioning them here just to confirm they're part of the plan, not because I have anything to report on them yet.

10.2 Introduction to the Spaceship Titanic Project

The actual end goal of this whole project, per the Project Brief, is to work with the Spaceship Titanic dataset from Kaggle. Each row is a passenger, with columns including HomePlanet, CryoSleep, Cabin (which encodes deck, number, and side of the ship), Destination, Age, VIP status, and spending across RoomService, FoodCourt, ShoppingMall, Spa, and VRDeck. The target, “Transported”, is what the model needs to predict.

I haven't started any actual work on this dataset yet, which lines up with the Plan of Action scheduling it for Week 7. That said, having now gone through Loan Status Prediction (Section 9.3), I can already picture roughly how this will go check missing values across both the categorical columns (HomePlanet, CryoSleep, Cabin, Destination, VIP) and numeric ones (Age and the spending columns), encode the categorical ones (with a real decision to make about Label Encoding vs one-hot for unordered ones like HomePlanet and Destination), maybe pull extra features out of Cabin since it seems to pack multiple pieces of info into one string, scale the numeric features, and then start with Logistic Regression and SVM before moving on to whatever I've learned from Decision Trees and ensembles by then.

10.3 Plan for Weeks 5–8

Going by the Plan of Action, here's roughly what's left. Week 5 (“Applied Machine Learning & Case Studies”) covers more complete end-to-end workflows, feature engineering, a first look at clustering and unsupervised learning, and comparing multiple models head-to-head. Week 6 (“Deep Learning & Reinforcement Learning Basics”) introduces neural networks and a bit of reinforcement learning.

Week 7 (“Final Project Development”) is where the actual Spaceship Titanic work happens EDA and preprocessing on this specific dataset, feature engineering, training and comparing multiple classification models, cross-validation and hyperparameter tuning (Section 8.2), a Kaggle submission, and even a small Streamlit app for the model, which I'm honestly a little nervous about since I've never touched Streamlit before, but also kind of looking forward to since it means having something interactive to actually show at the end.

Week 8 (“Finalisation & End-Term Submission”) is for refining the model based on the Kaggle leaderboard, polishing the deployment, cleaning up the code repository, and writing the End-Term report.

11. Conclusion

Looking back at these four weeks, the biggest shift for me has been going from treating ML as a kind of black box to actually understanding, at least at a working level, what's happening inside models like Linear Regression and Logistic Regression that they're really just an equation, a way of measuring how wrong that equation currently is, and an algorithm (gradient descent) that nudges the equation's parameters to make it a bit less wrong. Written down like that it sounds almost too simple, but it genuinely wasn't obvious to me four weeks ago.

The mini-projects, Loan Status especially, also taught me that real datasets basically never come clean there's almost always some mix of missing values, categorical columns that need encoding, and features on wildly different scales, all of which has to be sorted out before a model can even be trained. This is the part of the last four weeks that I think will transfer most directly to the Spaceship Titanic dataset.

At the same time, I know there's a fair amount I've only scratched the surface of statistics and probability (Section 2.4), the SVM kernel stuff, cross-validation and hyperparameter tuning in practice, and obviously Decision Trees, ensembles, and everything from Week 5 onward. My plan for what's left of this week is to actually finish the SVM material properly, then move into Decision Trees and ensembles using the textbook alongside whatever exercises are available, so that by the End-Term report I've got real implementations and results to show instead of just descriptions of what I've read.

References

1) Machine Learning video lecture series and accompanying implementation exercises, used as the primary study resource for Weeks 1–4.
2) Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd Edition, O'Reilly Media.
3) UCI Machine Learning Repository Heart Disease, Sonar (Rock vs Mine), and Pima Indians Diabetes datasets. Available: https://archive.ics.uci.edu/
4) Housing Price Prediction dataset, as used in the House Price Prediction mini-project of the applied machine learning exercises.
5) "Loan Prediction Dataset," Kaggle/Analytics Vidhya. Available via Kaggle dataset search.
6) Kaggle, "Spaceship Titanic" Competition. Available: https://www.kaggle.com/competitions/spaceship-titanic
7) scikit-learn developers, scikit-learn Documentation. Available: https://scikit-learn.org/stable/documentation.html
8) NumPy Developers, NumPy Documentation. Available: https://numpy.org/doc/

9) The pandas development team, pandas Documentation. Available: https://pandas.pydata.org/docs/

10) CS03 – Artificial Intelligence and Machine Learning, Plan of Action (PoA), Summer of Science 2026, IIT Bombay (Mentee: Mohit Khyalia, 24B2289; Mentor: Addepalli Bhargava Sarma).